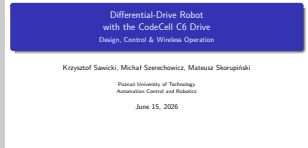
Differential-Drive Robot with the CodeCell C6 Drive

Design, Control & Wireless Operation

Krzysztof Sawicki, Michał Szerechowicz, Mateusz Skorupiński

Poznań University of Technology
Automation Control and Robotics

June 15, 2026



Introduce the project as a compact differential-drive robot built around the CodeCell C6 Drive board. Mention that the presentation covers the mechanical layout, electronics, control software, wireless interface, and safety mechanisms.

Outline

1. Project Overview
2. Hardware
3. Software Architecture
4. Movement Control
5. Wireless Communication
6. Control Application
7. Safety & Robustness
8. Summary & Future Work

Outline
Project Overview
Hardware
Software Architecture
Movement Control
Wireless Communication
Control Application
Safety & Robustness
Summary & Future Work

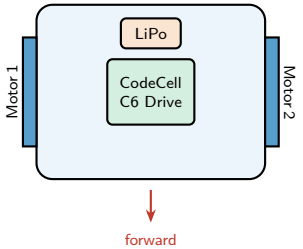
Use this slide only as orientation. Briefly tell the audience that the talk moves from goals and hardware, through control software and communication, and ends with safety and future improvements.

Project Goals

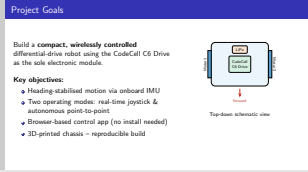
Build a **compact, wirelessly controlled** differential-drive robot using the CodeCell C6 Drive as the sole electronic module.

Key objectives:

- Heading-stabilised motion via onboard IMU
- Two operating modes: real-time joystick & autonomous point-to-point
- Browser-based control app (no install needed)
- 3D-printed chassis – reproducible build

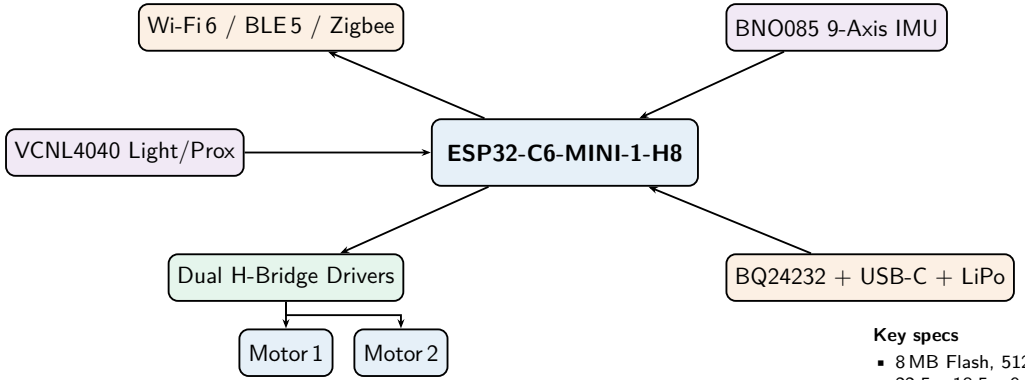


Top-down schematic view



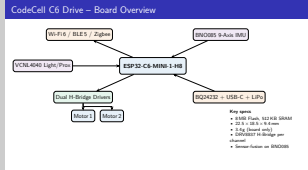
Explain that the goal was to build a small mobile robot with as little extra electronics as possible. Emphasize the two operating modes: real-time joystick control for manual driving, and target mode for point-to-point autonomous movement. Point out that the browser app makes the system easy to use from a phone or laptop.

CodeCell C6 Drive – Board Overview



Key specs

- 8 MB Flash, 512 KB SRAM
- $22.5 \times 18.5 \times 9.4$ mm
- 3.4 g (board only)
- DRV8837 H-Bridge per channel
- Sensor-fusion on BNO085



Describe the CodeCell C6 Drive as the core hardware module. Highlight the ESP32-C6 for computation and wireless communication, the BNO085 IMU for heading estimation, the motor drivers for the wheels, and the LiPo/USB-C power path. Mention that integrating these functions on one board reduces wiring and improves reproducibility.

Mechanical Design

Differential-drive layout

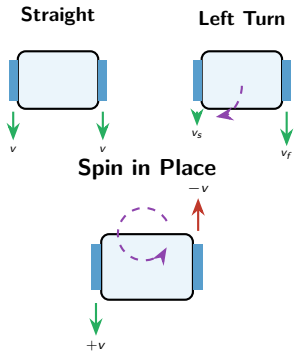
- Left motor → left wheel
- Right motor → right wheel
- Board and LiPo mounted centrally

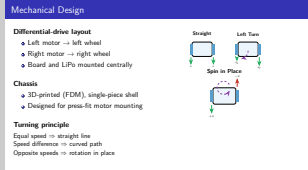
Chassis

- 3D-printed (FDM), single-piece shell
- Designed for press-fit motor mounting

Turning principle

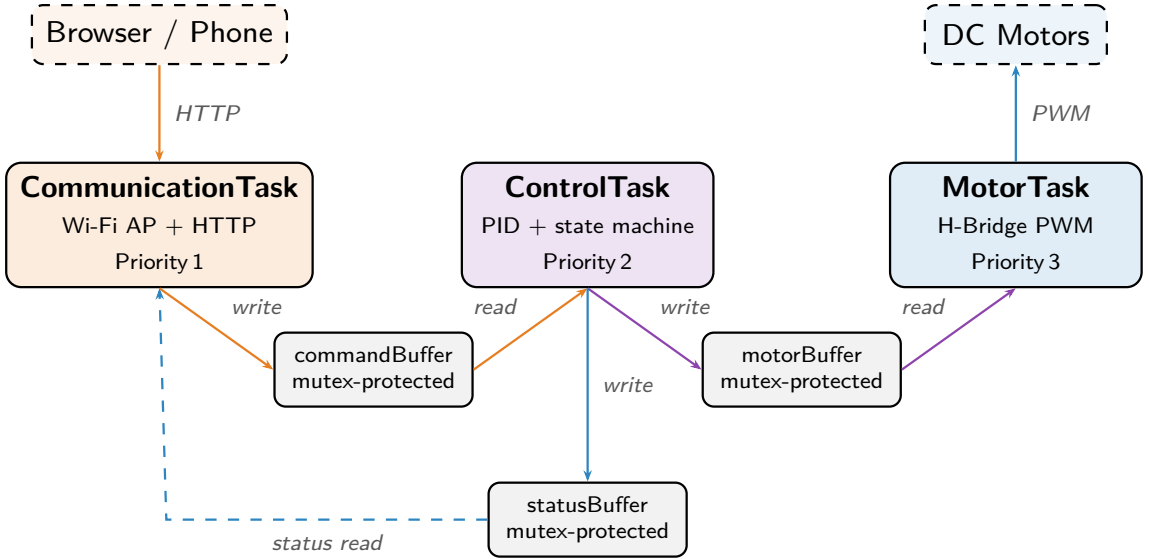
Equal speed ⇒ straight line
Speed difference ⇒ curved path
Opposite speeds ⇒ rotation in place

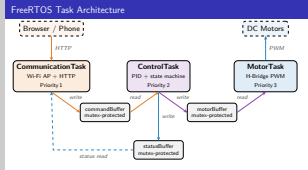




Explain the differential-drive principle: equal wheel speeds produce straight movement, different speeds produce a curve, and opposite speeds rotate the robot in place. Mention that the chassis is 3D printed and designed to keep the board, battery, and motors compact and centered.

FreeRTOS Task Architecture





Explain the firmware split into three FreeRTOS tasks. The communication task handles HTTP requests, the control task interprets commands and computes motor targets, and the motor task drives the H-bridges. Point out that the shared buffers use mutex protection so the tasks do not corrupt each other's data.

Differential-Drive Motor Mixing

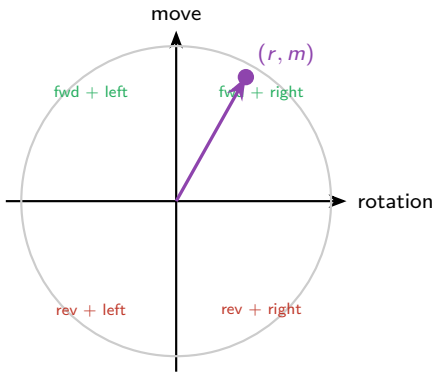
Joystick inputs in $[-100, 100]$:

- `move`: forward/backward
- `rotation`: turning

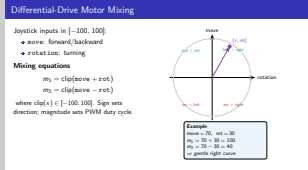
Mixing equations

$$m_1 = \text{clip}(\texttt{move} + \texttt{rot})$$
$$m_2 = \text{clip}(\texttt{move} - \texttt{rot})$$

where $\text{clip}(x) \in [-100, 100]$. Sign sets direction; magnitude sets PWM duty cycle.



Example
move $= 70$, rot $= 30$
$m_1 = 70 + 30 = 100$
$m_2 = 70 - 30 = 40$
$\Rightarrow$ gentle right curve



Describe how joystick input becomes left and right motor commands. The forward value is combined with the rotation value: adding rotation to one side and subtracting it from the other creates turning. Use the example to show that one wheel receives a stronger command, so the robot follows a gentle curve.

Heading-Stabilised Control – PID on Yaw

The BNO085 provides fused **yaw** (heading) at each control cycle (∼50 Hz). A discrete PID controller minimises heading error and feeds its output as the `rotation` input to the motor mixer.

Tuning parameters

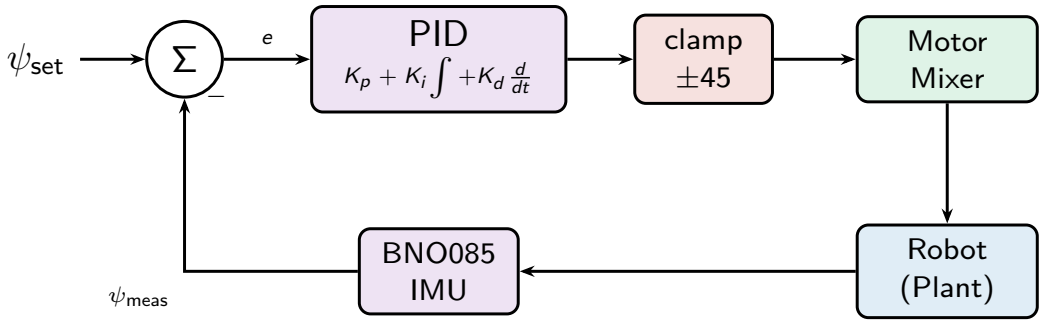
Parameter	Value
K_p	0.30
K_i	0.02
K_d	0.02
Integral limit	± 120
Max correction	± 45

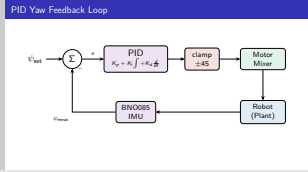
Error is wrapped to $(-180°, 180°]$ so the controller always uses the shortest rotation path.

Heading-Stabilised Control – PID on Yaw
The BNO085 provides fused **yaw** (heading) at each control cycle (∼50 Hz). A discrete PID controller minimises heading error and feeds its output as the `rotation` input to the motor mixer.
Tuning parameters
Parameter Value
K_p 0.30
K_i 0.02
K_d 0.02
Integral limit ± 120
Max correction ± 45
Error is wrapped to $(-180°, 180°]$ so the controller always uses the shortest rotation path.

Explain that the BNO085 provides a fused yaw angle at the control rate. The controller compares desired heading with measured heading and corrects drift through the rotation input. Mention that the tuning values are conservative so the robot corrects heading without aggressive oscillation.

PID Yaw Feedback Loop





Walk through the loop from left to right. The set heading is compared with measured heading, the PID produces a correction, the correction is clamped, and the motor mixer applies it to the wheels. The robot movement changes the measured yaw, closing the feedback loop through the IMU.

Control Modes – Direct vs. Target

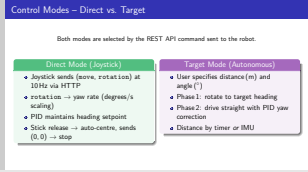
Both modes are selected by the REST API command sent to the robot.

Direct Mode (Joystick)

- Joystick sends (`move`, `rotation`) at 10 Hz via HTTP
- `rotation` → yaw rate (degrees/s scaling)
- PID maintains heading setpoint
- Stick release → auto-centre, sends $(0, 0)$ → stop

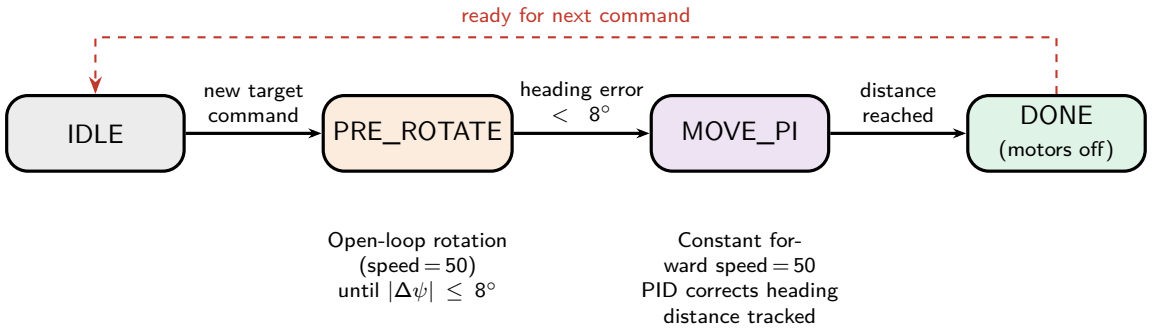
Target Mode (Autonomous)

- User specifies distance (m) and angle (°)
- Phase 1: rotate to target heading
- Phase 2: drive straight with PID yaw correction
- Distance by timer *or* IMU



Compare the two control modes. Direct mode is for manual driving and continuously receives joystick commands. Target mode is more autonomous: the user gives distance and angle, and the robot executes the movement in phases.

Target Mode – Two-Phase Execution



Distance estimation: time-based calibration or IMU double-integration.

Explain target mode as a simple state machine. First, the robot rotates until the heading error is below the threshold. Then it drives forward while PID keeps the heading stable. When the distance is reached, the motors stop and the robot returns to idle for the next command.

Start Ramp & Motor Deadband Compensation

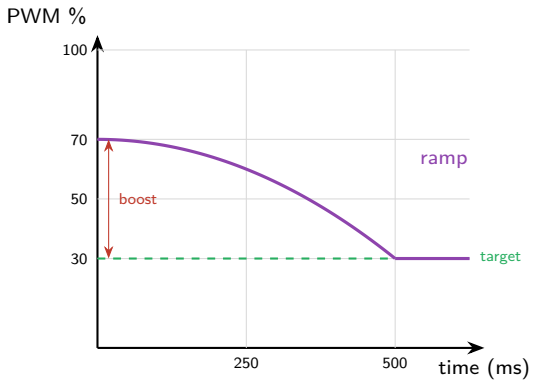
Small DC motors need extra torque to overcome **static friction** at startup.

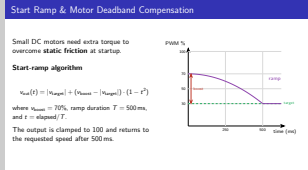
Start-ramp algorithm

$$v_{\text{out}}(t) = |v_{\text{target}}| + (v_{\text{boost}} - |v_{\text{target}}|) \cdot (1 - t^2)$$

where $v_{\text{boost}} = 70\%$, ramp duration $T = 500\,\text{ms}$, and $t = \text{elapsed}/T$.

The output is clamped to 100 and returns to the requested speed after 500 ms.





Explain that small DC motors often need extra torque to start moving because of static friction. The start ramp briefly applies a higher boost and then decays smoothly to the requested speed. This improves repeatability without permanently overpowering the motors.

Wi-Fi Access Point & REST API

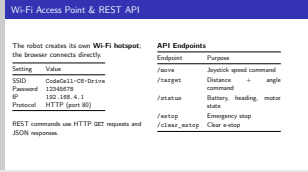
The robot creates its own **Wi-Fi hotspot**; the browser connects directly.

Setting	Value
SSID	`CodeCell-C6-Drive`
Password	`12345678`
IP	`192.168.4.1`
Protocol	HTTP (port 80)

REST commands use HTTP `GET` requests and JSON responses.

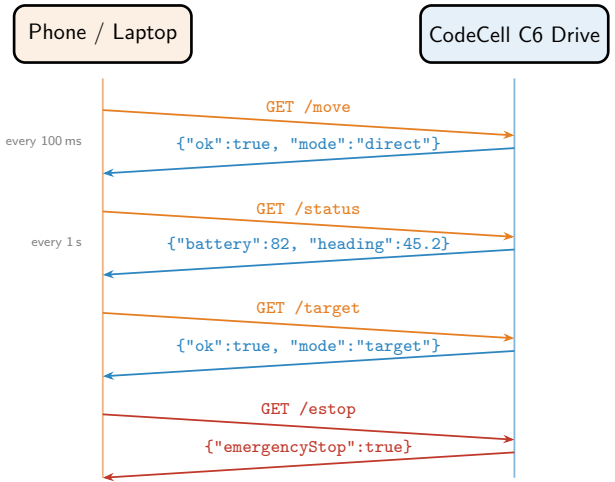
API Endpoints

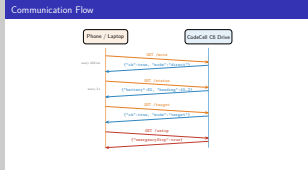
Endpoint	Purpose
`/move`	Joystick speed command
`/target`	Distance + angle command
`/status`	Battery, heading, motor state
`/estop`	Emergency stop
`/clear_estop`	Clear e-stop



Describe the communication setup. The robot creates its own Wi-Fi access point, so no external router is required. The browser sends simple HTTP GET requests for movement, target mode, status, emergency stop, and clearing the stop condition.

Communication Flow





Use this slide to explain the message sequence. Movement commands are sent frequently, status is polled less often, target commands start autonomous motion, and emergency stop has a dedicated endpoint. Emphasize that the API is simple enough to test from any browser.

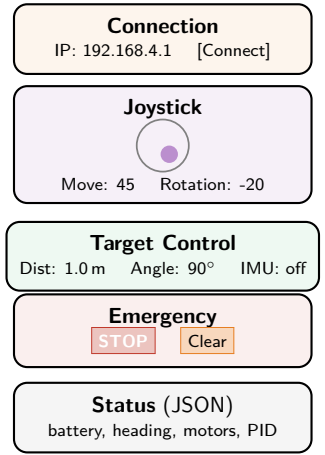
Web-Based Control Interface

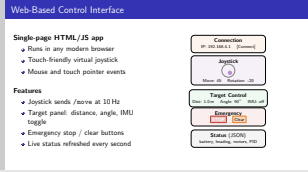
Single-page HTML/JS app

- Runs in any modern browser
- Touch-friendly virtual joystick
- Mouse and touch pointer events

Features

- Joystick sends `/move` at 10 Hz
- Target panel: distance, angle, IMU toggle
- Emergency stop / clear buttons
- Live status refreshed every second





Explain that the control app is a single-page HTML and JavaScript interface. It works on a phone or laptop and supports mouse and touch input. Mention the main controls: connection, joystick, target command panel, emergency stop, and live status.

Safety Mechanisms

Emergency Stop

- One-click E-Stop via `/estop`
- All tasks check one shared flag
- Cleared only by explicit `/clear_estop`

Command Timeout

- Every command is timestamped
- No update within **300 ms** $\rightarrow$ stop
- Handles Wi-Fi or browser failure

Thread Safety

- Three FreeRTOS mutexes guard shared buffers
- No direct cross-task memory access
- Sequence counter detects new commands

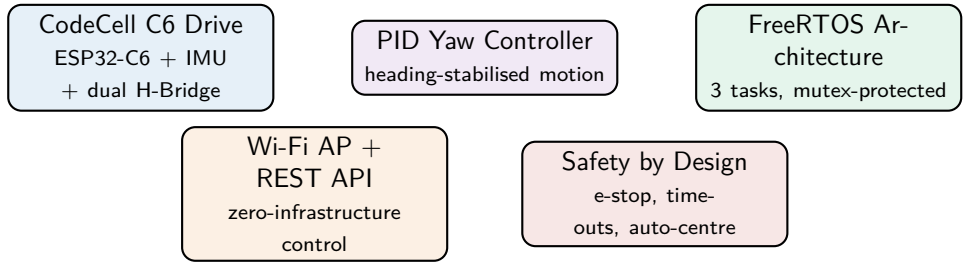
Auto-Centre Joystick

- On `pointerup`, joystick snaps to $(0, 0)$
- Sends stop command immediately
- Prevents unintended continued motion

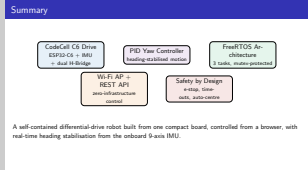
Safety Mechanisms
Emergency Stop
- One-click E-Stop via /estop
- All tasks check one shared flag
- Cleared only by explicit /clear_estop
Command Timeout
- Every command is timestamped
- No update within 300 ms → stop
- Handles Wi-Fi or browser failure
Thread Safety
- Three FreeRTOS mutexes guard shared buffers
- No direct cross-task memory access
- Sequence counter detects new commands
Auto-Centre Joystick
- On pointerup, joystick snaps to (0,0)
- Sends stop command immediately
- Prevents unintended continued motion

Present safety as part of the core design. Emergency stop overrides normal motion, command timeout stops the robot if communication is lost, mutexes protect shared state, and joystick auto-centering prevents continued movement after input release.

Summary



A self-contained differential-drive robot built from one compact board, controlled from a browser, with real-time heading stabilisation from the onboard 9-axis IMU.



Summarize the project as a self-contained differential-drive robot with integrated wireless control, motor driving, and IMU feedback. Reinforce the main technical achievements: FreeRTOS architecture, PID heading control, browser control, and safety mechanisms.

Future Work

- **Obstacle avoidance** – reactive stop / turn behaviours
- **Odometry improvements** – wheel encoders or visual odometry
- **Multi-waypoint navigation** – sequence of target commands
- **BLE / Zigbee control** – lower-power communication options
- **Smartphone app** – richer UI and persistent settings

Future Work
- **Obstacle avoidance** – reactive stop / turn behaviours
- **Odometry improvements** – wheel encoders or visual odometry
- **Multi-waypoint navigation** – sequence of target commands
- **BLE / Zigbee control** – lower-power communication options
- **Smartphone app** – richer UI and persistent settings

Discuss practical extensions. Obstacle avoidance could use the proximity sensor, odometry could improve distance estimation, multi-waypoint navigation would make target mode more useful, and BLE/Zigbee or a smartphone app could improve the control experience.

Thank you

Questions?

Source code & 3D models available in the project repository

Thank you
Questions?
Source code & 3D models available in the project repository

Close by inviting questions. Be ready to answer about PID tuning, communication reliability, mechanical design choices, and how the robot estimates distance in target mode.